Printix MCP — User Manual

> **Version:** 6.7.116 **Audience:** Administrators, helpdesk staff, and power users who interact with the Printix MCP Server through an AI assistant (claude.ai, ChatGPT, Claude Desktop).
> **Language:** English · German version: `MCP_MANUAL_DE.pdf`

What is Printix MCP?

The Printix MCP Server is a bridge between modern AI assistants and the Printix Cloud Print API. It exposes **over 100 tools** that let you drive Printix in natural language — from a simple "*Which printers do we have in Düsseldorf?*" all the way to "*Show me the 10 most expensive printers of last quarter versus the quarter before*".

You do **not** need to memorize tool names. The assistant picks the right tool from your question. This manual gives you an overview of *what* is possible, so you know what to ask.

How to read this manual

Each category starts with a short intro, followed by a tool table and **example dialogues**. The dialogues show a typical user prompt and which tool the assistant calls internally. Feel free to copy the prompts verbatim or use them as inspiration.

Table of Contents

1. System & Self-Diagnostics
2. Printers, Sites & Networks
3. Print Jobs & Cloud Print
4. Users, Groups & Workstations
5. Cards & Card Profiles
6. Reports & Analytics
7. Report Templates & Scheduling
8. Capture / Workflow Automation
9. Operations, Maintenance & Audit

---

1. System & Self-Diagnostics

These tools answer the meta questions: *Who am I? Is everything running? What is my role? What should I do next?* Ideal as an opener for a new session — or when something unexpectedly fails and you need to know **why**.

Tool	Purpose
`printix_status`	Health check: server running, tenant reachable, API versions.
`printix_whoami`	Shows the currently connected tenant and your own Printix user.
`printix_tenant_summary`	Compact overview: printer / user / site / card count and open jobs.
`printix_explain_error`	Translates a Printix error code or message into plain language plus a suggested fix.
`printix_suggest_next_action`	Suggests a sensible next step based on a context string.
`printix_natural_query`	Takes a natural-language question and suggests the right reporting tool.

Example dialogues

Prompt: "*Is everything OK with Printix?*" The assistant calls `printix_status` and reports tenant, API connectivity, and versions.

Prompt: "*Who am I logged in as on Printix?*" `printix_whoami` returns tenant name, your e-mail, and admin flag.

Prompt: "*Give me a snapshot of my Printix tenant.*" `printix_tenant_summary` produces a one-glance KPI block — perfect as a conversation opener.

Prompt: "*What does 'Job submission failed — 403' mean?*" `printix_explain_error` explains the code and names typical causes (missing scope, wrong tenant ID, expired token).

---

2. Printers, Sites & Networks

Everything around physical and logical infrastructure: printers, queues, sites, networks, and SNMP configuration. Read and write operations. The `*_context` tools return aggregated views (e.g. "queue + printer + recent jobs" in one call), saving the assistant multiple round-trips.

Tool	Purpose
`printix_list_printers`	Lists all printers (optional search term).
`printix_get_printer`	Details and capabilities of one printer.
`printix_resolve_printer`	Finds the best printer via fuzzy match (name + location + model).
`printix_network_printers`	All printers of a network or site.
`printix_get_queue_context`	Queue + printer object + recent jobs in a single call.
`printix_printer_health_report`	Printer status overview: online / offline / error states.
`printix_top_printers`	Top-N printers by print volume (days, limit, metric).
`printix_list_sites`	All sites of the tenant.
`printix_get_site`	Site details.
`printix_create_site` / `printix_update_site` / `printix_delete_site`	Site management.
`printix_site_summary`	Site + networks + printers in an aggregated block.
`printix_list_networks`	Networks, optionally filtered by site.
`printix_get_network`	Network details.
`printix_create_network` / `printix_update_network` / `printix_delete_network`	Network management.
`printix_get_network_context`	Network + its site + printers in a single block.
`printix_list_snmp_configs` / `printix_get_snmp_config`	SNMP configurations.
`printix_create_snmp_config` / `printix_delete_snmp_config`	Create / delete an SNMP config.
`printix_get_snmp_context`	SNMP config + affected printers + network in one block.

Example dialogues

Prompt: "*Which Brother printers are in Düsseldorf?*" `printix_resolve_printer("Brother Düsseldorf")` — token-based fuzzy match returns every device where both tokens appear anywhere in name / location / vendor / site.

Prompt: "*Show me all printers in network 9cfa4bf0.*" `printix_network_printers(network_id="9cfa4bf0")` — when the API does not provide a

direct network→printer mapping, the tool resolves the network's site internally and returns the relevant printers. The `resolution_strategy` field shows which path was taken.

Prompt: "*Give me a full summary of the DACH site.*" `printix_site_summary(site_id=…)` — site metadata, all networks with counters, and all printers in one block.

Prompt: "*Which printers are currently offline?*" `printix_printer_health_report` groups by status and surfaces the problem devices first.

---

3. Print Jobs & Cloud Print

Inspect jobs, submit them, delegate to other users. Contains the productive shortcuts (`quick_print`, `send_to_user`) that bundle the typical 3-step submit → upload → complete flow into a single call.

Tool	Purpose
`printix_list_jobs`	All jobs, optionally filtered to a queue.
`printix_get_job`	Details of one job.
`printix_submit_job`	Submit a print job (step 1 of the 3-step submit flow).
`printix_complete_upload`	Complete the upload and release the job.
`printix_delete_job`	Cancel a job.
`printix_change_job_owner`	Change the job owner (delegation).
`printix_jobs_stuck`	Jobs that have been hanging longer than N minutes.
`printix_quick_print`	One-shot print: URL + recipient → done.
`printix_send_to_user`	Forward a document directly to another user.

Example dialogues

Prompt: "*Send this PDF to marcus@company.com as secure print.*" `printix_quick_print(recipient_email="marcus@company.com", file_url=…, filename="contract.pdf")` — submit + upload + complete in one call.

Prompt: "*Which print jobs have been stuck for more than 30 minutes?*" `printix_jobs_stuck(minutes=30)` lists blocked jobs with age and owner.

Prompt: "*Hand over job 4711 to marcus@company.com — I'm going on vacation.*" `printix_change_job_owner(job_id="4711", new_owner_email="marcus@company.com")`.

---

4. Users, Groups & Workstations

Full lifecycle: create, edit, disable, diagnose. `user_360` and `diagnose_user` are especially valuable for helpdesk work: they pull user master data, groups, cards, workstations, SSO status, and recent print activity into **one** response.

Tool	Purpose
`printix_list_users`	All users of the tenant with pagination and role filter.
`printix_get_user`	User details.
`printix_find_user`	Search by e-mail fragment or name.
`printix_user_360`	360° view: user + cards + groups + workstations + recent jobs.
`printix_diagnose_user`	Helpdesk diagnosis: what works, what doesn't, why.
`printix_create_user` / `printix_delete_user`	Create / delete user.
`printix_generate_id_code`	Generate a new ID code for a user (self-service token).
`printix_onboard_user` / `printix_offboard_user`	Guided on- and off-boarding (multiple steps in one call).
`printix_list_admins`	All admins of the tenant.
`printix_permission_matrix`	Matrix: user × permissions.
`printix_inactive_users`	Users who haven't printed in N days.
`printix_sso_status`	Check SSO mapping for an e-mail.
`printix_list_groups` / `printix_get_group`	List / detail groups.
`printix_create_group` / `printix_delete_group`	Group management.
`printix_list_workstations` / `printix_get_workstation`	List / detail workstations.

Example dialogues

Prompt: "*Tell me everything you know about marcus@company.com.*" `printix_user_360(query="marcus@company.com")` returns the full 360° view.

Prompt: "*Why can't Anna print anymore?*" `printix_diagnose_user(email="anna@company.com")` checks status, SSO, cards, groups, and active blockers — then returns a diagnosis with remediation hints.

Prompt: "*Which users have been inactive for 180 days?*" `printix_inactive_users(days=180)` — offboarding candidate list.

Prompt: "*Onboard a new employee: peter@company.com, Peter Miller, Finance group.*" `printix_onboard_user(...)` runs all steps in the correct order.

---

5. Cards & Card Profiles

Everything RFID / Mifare / HID: registration, mapping, profile detection, bulk import. The `decode` / `transform` tools are particularly useful when debugging unknown card profiles.

Tool	Purpose
`printix_list_cards`	Cards of a specific user.
`printix_list_cards_by_tenant`	All cards of the tenant (filter: `all` / `registered` / `orphaned`).
`printix_search_card`	Find a card by ID or card number.
`printix_register_card`	Assign a card to a user.
`printix_delete_card`	Remove the card assignment.
`printix_get_card_details`	Card + local mapping + owner details in one block.
`printix_decode_card_value`	Decode a raw card value (Base64, hex, YSoft/Konica variants).
`printix_transform_card_value`	Run a card value through the transformation chain (hex↔decimal, reverse, Base64, prefix/suffix …).
`printix_get_user_card_context`	User + all their cards + profiles in one block.
`printix_list_card_profiles` / `printix_get_card_profile`	List / detail card profiles.
`printix_search_card_mappings`	Search the local card-mapping DB.
`printix_bulk_import_cards`	CSV / JSON bulk import (with profile selection + dry-run).
`printix_suggest_profile`	Suggest a profile based on a sample UID (ranked top-10).
`printix_card_audit`	Audit trail of all card changes for a user.
`printix_find_orphaned_mappings`	Local mappings without a matching Printix user (cleanup candidates).

Example dialogues

Prompt: "*Which cards does Marcus have?*" `printix_list_cards` (after a `printix_find_user` for the user ID) or, more compact, `printix_get_user_card_context`.

Prompt: "*What's the card with UID* `04:5F:F0:02:AB:3C` *?*"
`printix_decode_card_value(card_value="04:5F:F0:02:AB:3C")` recognizes hex-UID with separators, returns `decoded_bytes_hex` and `profile_hint: "hex-input"`.

Prompt: "*Import 500 cards from this CSV — but first as a dry-run.*"
`printix_bulk_import_cards(..., dry_run=True)` validates each row against the selected profile and shows preview values without touching Printix.

Prompt: "*For UID* `045FF002` *— which profile matches?*"
`printix_suggest_profile(sample_uid="045FF002")` returns the top-10 profiles with a score plus the `best_match`.

---

6. Reports & Analytics

The reporting engine runs against a separate SQL Server warehouse. You get KPIs, trends, anomalies, and ad-hoc queries through one unified interface. `query_any` is the universal entry point — the specialised tools are fast shortcuts for common questions.

Tool	Purpose
`printix_reporting_status`	Status of the reporting engine (DB connectivity, last nightly runs, preset count).
`printix_query_any`	Universal: provide a preset + filters, get a table.
`printix_query_print_stats`	Print volume by arbitrary dimension.
`printix_query_cost_report`	Print cost, optionally by department / user.
`printix_query_top_users` / `printix_query_top_printers`	Top-N with time window.
`printix_query_anomalies`	Anomaly detection (outliers, unusual patterns).
`printix_query_trend`	Trend lines over time.
`printix_query_audit_log`	Structured audit trail of the MCP server itself (actions, objects, actor).
`printix_top_printers` / `printix_top_users`	Shortcut version (days + limit + metric).
`printix_print_trends`	Trend by day / week / month.
`printix_cost_by_department`	Cost aggregated per department.
`printix_compare_periods`	Period A vs. period B delta.

Example dialogues

Prompt: "*Who printed the most last month?*" `printix_top_users(days=30, limit=10, metric="pages")`.

Prompt: "*What's the print trend over the last 90 days, monthly?*" `printix_print_trends(group_by="month", days=90)`.

Prompt: "*Compare the last 30 days to the 30 before — what changed?*" `printix_compare_periods(days_a=30, days_b=30)` returns delta KPIs.

Prompt: "*Which actions did user X take in the MCP on April 15?*" `printix_query_audit_log(start_date="2026-04-15", end_date="2026-04-15", ...)` — filtered to the current tenant.

---

7. Report Templates & Scheduling

When you need an analysis repeatedly: save as a template, schedule for recurring delivery, send by e-mail. Design options (colours, logos, layout) are listed via `list_design_options`; `preview_report` renders a preview PDF without actually delivering.

Tool	Purpose
`printix_save_report_template`	Save a query + design as a template.
`printix_list_report_templates`	All saved templates.
`printix_get_report_template`	Template details.
`printix_delete_report_template`	Delete a template.
`printix_run_report_now`	Run a template once and deliver.
`printix_send_test_email`	Test mail to an address for SMTP verification.
`printix_schedule_report`	Schedule a template as a cron job.
`printix_list_schedules`	All active schedules.
`printix_update_schedule` / `printix_delete_schedule`	Edit / delete a schedule.
`printix_list_design_options`	Available colour schemes, logos, layout variants.
`printix_preview_report`	Preview PDF of a report without delivery.

Example dialogues

Prompt: "*Save the current top-10 user report as template 'Monthly Print Top-10'.*" `printix_save_report_template(...)` stores the preset + filter + design.

Prompt: "*Deliver this template on the first business day of every month to management@company.com.*" `printix_schedule_report(report_id=…, cron="0 8 1 * *", recipients=["management@company.com"])`.

Prompt: "*Show me a preview PDF of template XY.*" `printix_preview_report(report_id=…)` renders the PDF without delivering anything.

---

8. Capture / Workflow Automation

Capture connects scanned documents to target systems (Paperless-ngx, SharePoint, DMS …) through plugins. The tools here show status and configured profiles — the actual plugin configuration happens in the web UI.

Tool	Purpose
`printix_list_capture_profiles`	All capture profiles configured for the tenant.
`printix_capture_status`	Status: server port, webhook base URL, available plugins, configured profiles.

Example dialogues

Prompt: "*Is capture active, and which plugins are installed?*" `printix_capture_status` returns the plugin list (including paperless_ngx) and the number of configured profiles.

Prompt: "*Which capture profiles are active on my tenant?*" `printix_list_capture_profiles` — list with target system, filename pattern, and recent runs.

---

9. Operations, Maintenance & Audit

Backups, demo data, feature requests, audit log. A mix of operations (backup) and meta (feature tracking).

Tool	Purpose
`printix_list_backups`	All existing backups.
`printix_create_backup`	Create a new backup of the local config + DB.
`printix_demo_setup_schema`	Create the demo schema in the reporting DB (sandbox).
`printix_demo_generate`	Generate synthetic demo data.
`printix_demo_rollback`	Remove demo data again (by demo tag).
`printix_demo_status`	Shows which demo sets are currently active.
`printix_list_feature_requests` / `printix_get_feature_request`	Ticketing system for feature requests.

Example dialogues

Prompt: "*Take a backup before I change something.*" `printix_create_backup` produces a ZIP with DB + config + metadata.

Prompt: "*Set up a demo environment with 50 users and 500 jobs.*" `printix_demo_setup_schema` (one-time) + `printix_demo_generate(users=50, jobs=500)`.

Prompt: "*List all open feature requests.*" `printix_list_feature_requests(status="open")`.

Tips for productive AI dialogues

1. **Talk in goals, not tools.** "Who prints too much?" beats "call query_top_users with days=30". The assistant picks the right tool.
2. **Give context.** "Marcus from Finance" is less ambiguous than just "Marcus".
3. **Leverage the 360° tools.** `printix_user_360`, `printix_get_queue_context`, `printix_site_summary` replace several follow-ups.
4. **Ask for the root cause.** "Why did this fail?" or "Explain this error" triggers `printix_explain_error` or `printix_diagnose_user`.
5. **Dry-run before bulk operations.** `printix_bulk_import_cards` has a `dry_run=True` mode — use it.